

Data Science

Lecture 04: Machine Learning

Prof. Dr. Michael Granitzer

Module 1

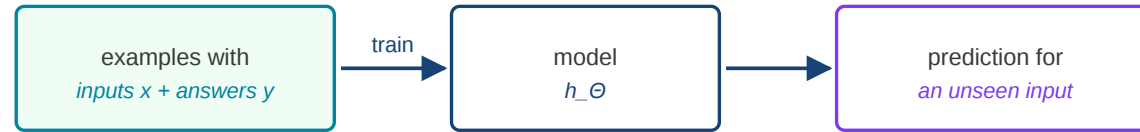
What Is a Learning Problem?

Guiding Question: *What has to be true of a task — and of the data — before "learn it from examples" is a sensible answer?*

Learning From Answers

A radiologist has labelled ten thousand scans. A bank has ten years of loans and knows which ones defaulted. In both cases **every example carries its answer**.

Supervised: every training example carries the answer



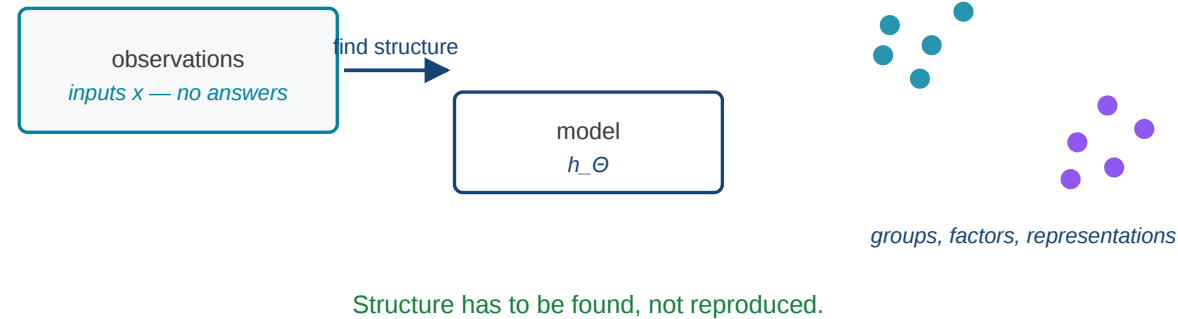
The learner is told the correct output for each case it sees.

What information does the learner receive during training?

Learning Without Answers

Now the same bank wants to know **what kinds of customer it has**. Nobody has ever written down the list — the categories are what we are looking for.

Unsupervised: no answers — only the observations themselves

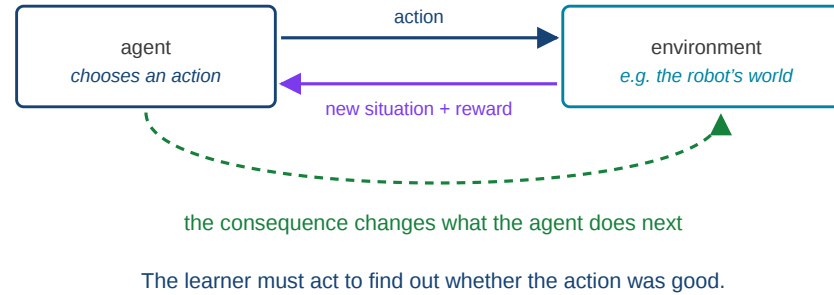


What can we learn when nobody tells us the correct answer?

Learning by Acting

A robot has to cross a room. Nobody can label the correct motor command for every situation it might find itself in — and it only discovers whether a move was good **after making it**.

Reinforcement: no answer is given — only the consequence of acting



How can a learner improve when it must act in order to find out whether the action was any good?

Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting



Same Goal, Different Information

Same goal, different information

	WHAT IT IS GIVEN	WHAT FEEDBACK IT GETS
Supervised	inputs + answers	the answer, every time
Unsupervised	inputs only	nothing
Reinforcement	the situation it acts in	a reward, after acting

The paradigms differ in **what the learner is given** and **what feedback it gets** — not in what we ultimately want from it.

What Is Common to All Three?

Nobody wrote the rule

The mapping from situation to answer is **too complex or too implicit** to specify by hand.

Examples exist

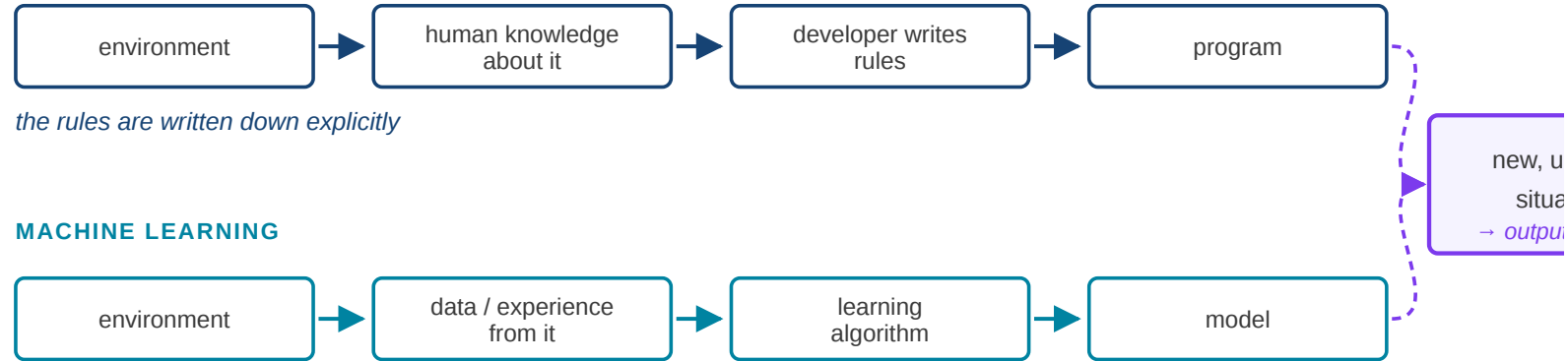
There is data — or experience — in which the desired behaviour is demonstrated, even if it was never described.

So what replaces the rule that nobody wrote?

Two Ways to Get Behaviour

Both are judged on situations neither of them has seen

TRADITIONAL PROGRAMMING



the rules are written down explicitly

MACHINE LEARNING



the rules are inferred from what the environment produced

What Actually Changed

TRADITIONAL PROGRAMMING

The developer's **knowledge becomes rules**. Deduction: rules plus input give the output.

The assumptions are visible — they are in the code.

MACHINE LEARNING

The environment's **data becomes a model**. Induction: examples plus a learning procedure give a rule.

The assumptions are still there — but now they are in the data, the model family and the objective.

The rules do not disappear when nobody writes them. They become **harder to inspect** — which is why the rest of this lecture is about making every part of that pipeline explicit.

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

What Would It Mean for a Machine to Learn?

Before any definition — three questions you would have to answer about *any* system before you could say it had learned anything:

- ? What is it supposed to get better at?
- ? What does it get to look at while getting better?
- ? How would we know whether it improved?

Task T

what the system should learn
to do

Experience E

what it learns from

Performance P

how we decide it improved

Chess



A program that plays, loses, and plays again — millions of times against itself.

- **Task** — play chess well
- **Experience** — games played against itself, and positions from expert play
- **Performance** — fraction of games won; rating against other engines

A Large Language Model



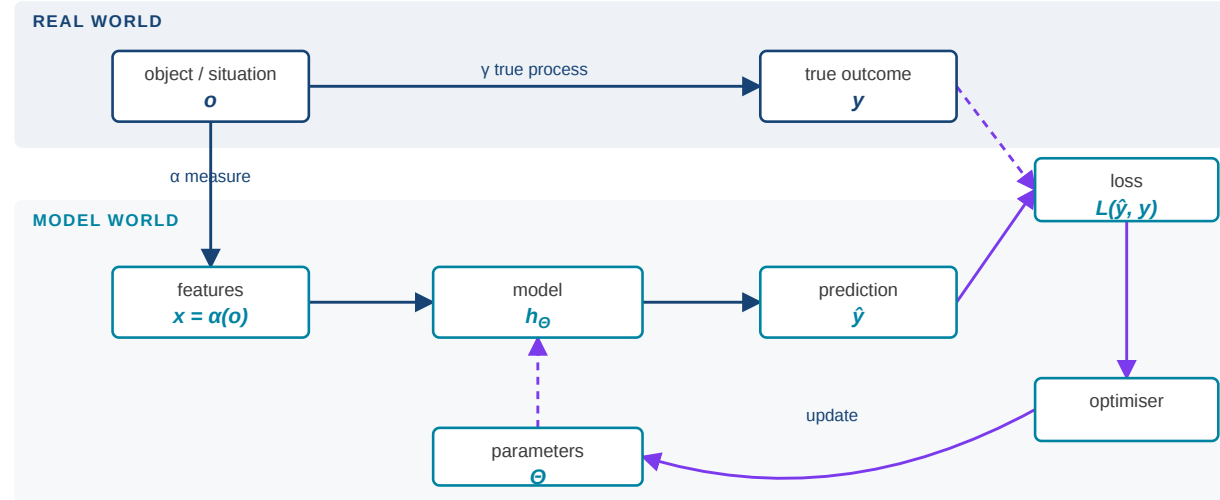
A model shown enormous quantities of text, asked only to continue it.

- **Task** — predict the next word
- **Experience** — a very large corpus of text, then human feedback
- **Performance** — likelihood of held-out text; later, human judgement

Two systems with nothing in common — a board game and a text corpus — fit the **same three-part description**. That is what makes it a definition rather than a description of one program.

What Would We Ideally Like to Know?

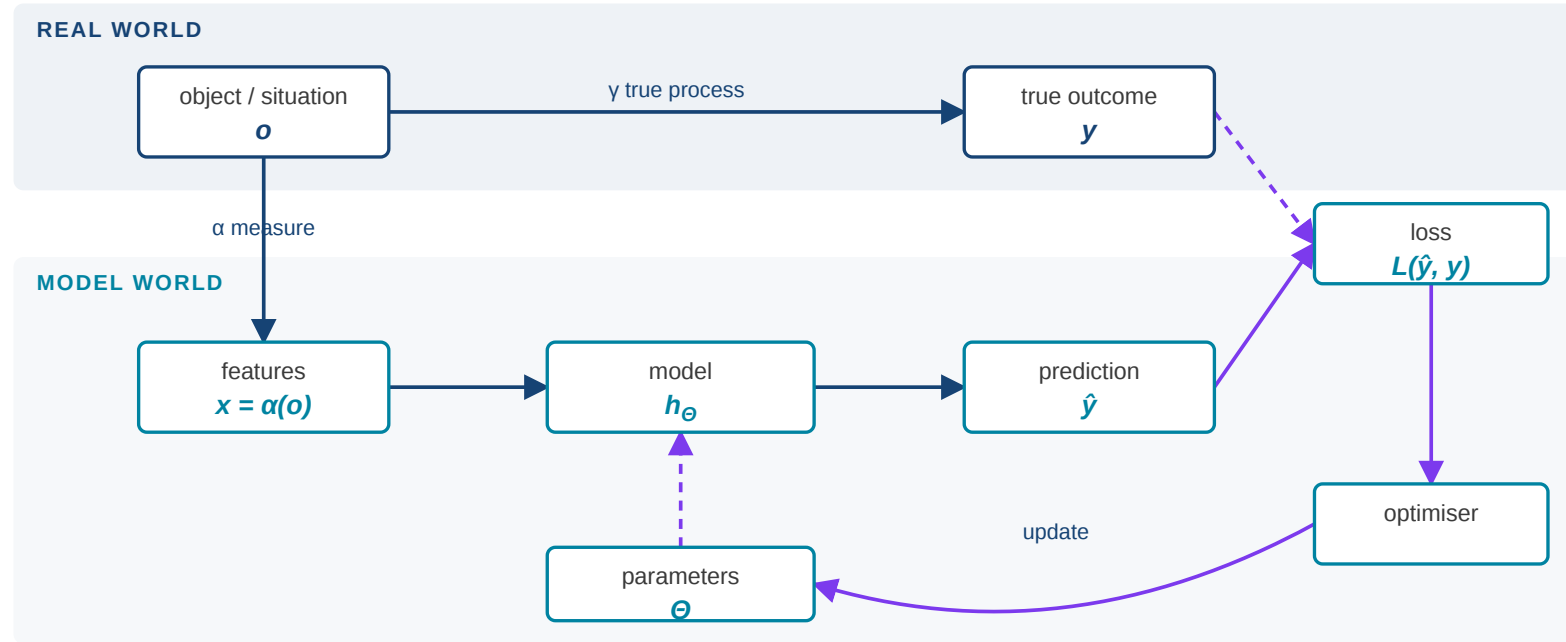
The optimiser changes the parameters — and the training loop closes



There is some real process that turns a situation into an outcome. Why can we not simply write it down?

From the Real World to a Model of It

The optimiser changes the parameters — and the training loop closes



Two Worlds, One Correspondence

$$\gamma(o); \rightsquigarrow; h_{\Theta}(\alpha(o)); \equiv; h_{\Theta}(x)$$

γ

the **real process** — unknown,
and not ours to write down

α

the **representation** — what
we chose to measure

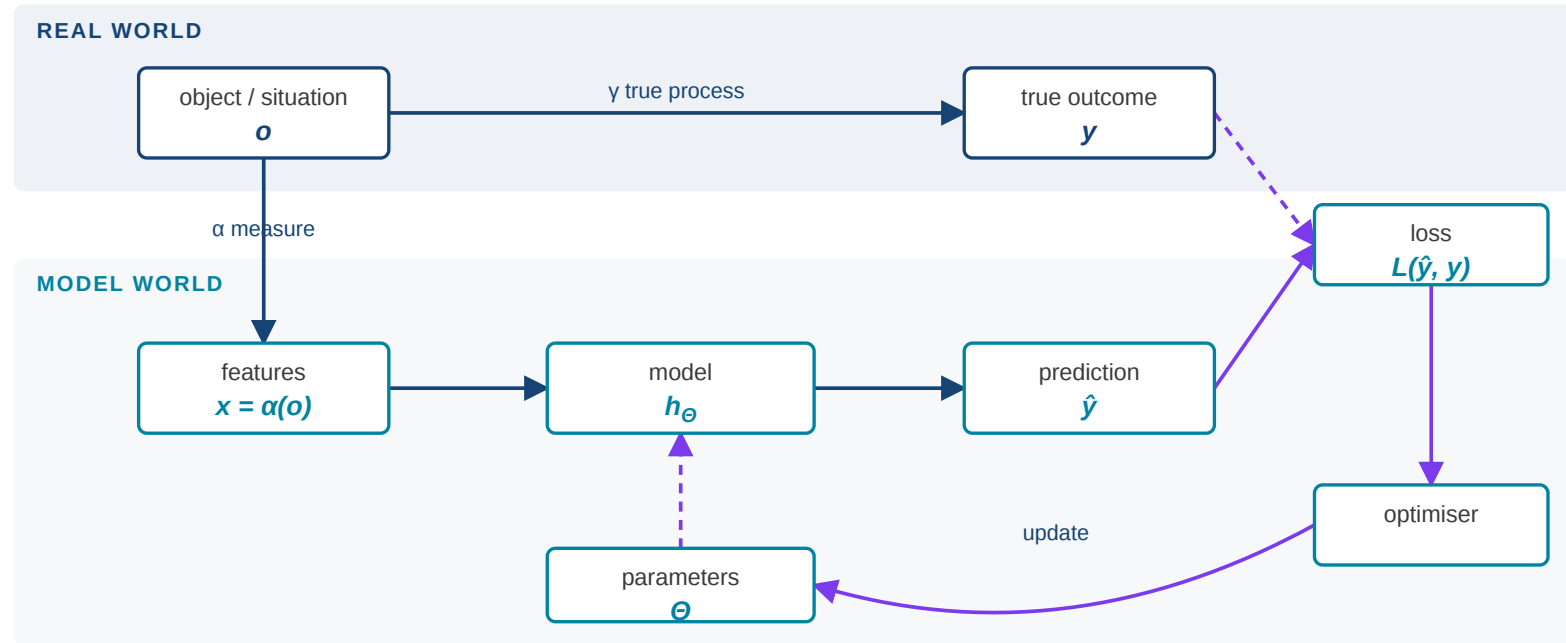
h_{Θ}

the **model** — our
approximation, with
parameters Θ

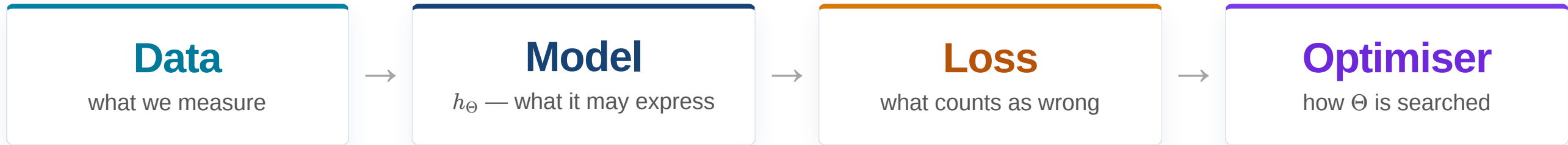
We never learn the world. We build a **model world** and approximate the process inside it.

Closing the Loop

The optimiser changes the parameters — and the training loop closes



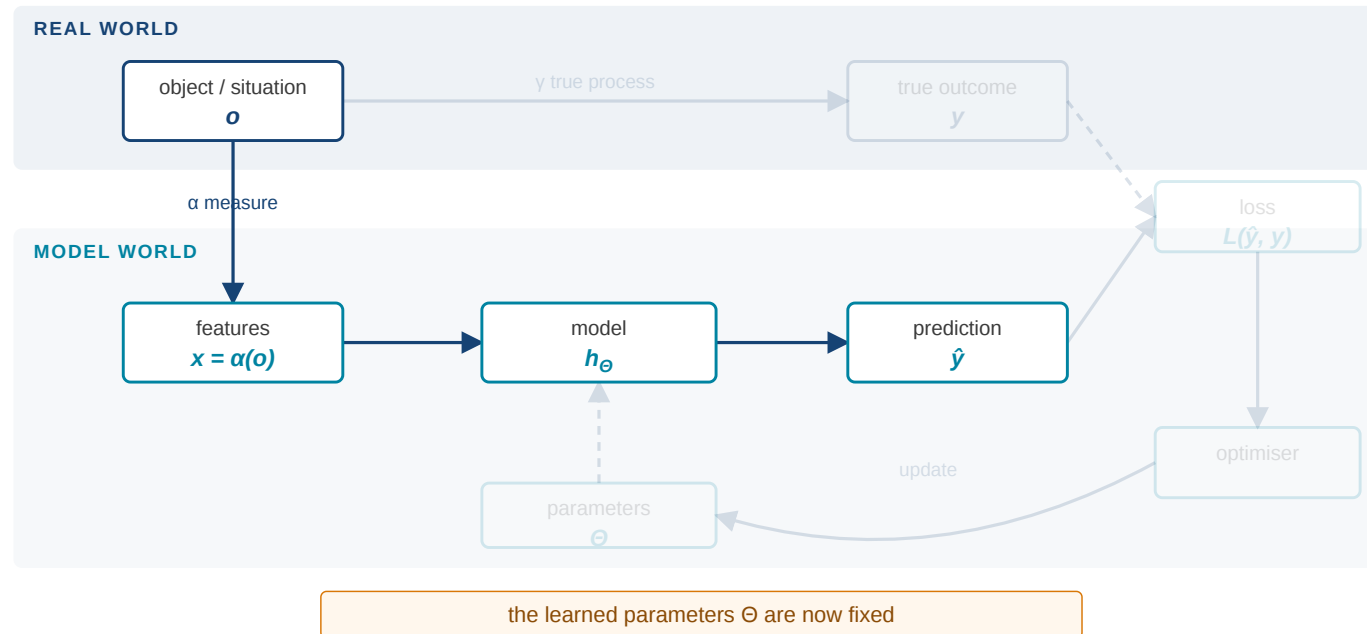
The Four Choices



The optimiser has settings of its own that are not learned from the data — **hyperparameters**. Choosing them is part of the outer process, not of the loop.

Training Is Not Using

Inference: a new, unseen situation follows one path — the training machinery is gone



Which of these boxes still exist when the system is actually deployed?

The Task: Predicting Acceleration

The world

O — vehicles

$Y = \mathbb{R}$ — time to accelerate to 100 km/h

$\gamma(o)$ — the true acceleration time of vehicle o

The model world

α — measure wheels, horsepower, weight, torque

X — those four numbers, for every vehicle

h_{Θ} — a linear function of them

The Data

wheels	horsepower	weight	torque	acceleration time
4	150	1350	250	9.8
4	320	1600	430	4.9
6	95	2100	210	16.2
...	...	...	...	...

The last column is y the rest is $\mathbf{x}$.

The Remaining Two Pieces

Loss function

Mean squared error — the average of the squared differences between predicted and observed acceleration times.

Learning algorithm

LMS — least mean squares, which adjusts the parameters to reduce that error.

Four choices — representation, model family, loss, optimiser — and the system is fully specified. Every method in this chapter is a different set of four.

A learning problem becomes concrete the moment you can name its **task, experience and performance measure**, and its system becomes buildable the moment you can name its **representation, model, loss and optimiser**.

Designing the Learning Problem

Before any model is chosen, four things have to be decided — and each of them is a claim about the world.

THE TASK

- ? What should the system predict or decide?
- ? What is one unit of prediction?
- ? What counts as success?

THE DATA

- ? Which observations represent the problem?
- ? How were the examples sampled?
- ? Will future cases look like these?

THE REPRESENTATION A

THE TARGET



? What are the inputs, concretely?

? Which features are available — and which are merely convenient?

? What is lost in the measurement?

? What is γ , in words a domain expert would accept?

? Is the label we have the thing we care about, or a proxy for it?

Designing the System

MODEL AND OBJECTIVE

- ? What does the model family assume, and how flexible is it?
- ? Does minimising this loss correspond to what we actually want?

LEARNING

- ? Which procedure searches the parameters?
- ? How sensitive is the result to initialisation and randomness?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

Designing the System — Evaluation

EVALUATION AND GENERALISATION

- ? Which metric matters, and which data is held out?
- ? How uncertain is the estimate of performance?
- ? What happens when the environment changes?

UNCERTAINTY AND USE

- ? Which uncertainty must be represented explicitly?
- ? Is the prediction informing a decision — and what do the errors cost?

These are the questions the rest of the course answers. Each lecture takes one or two of them and makes them precise.

Module 2

How Learning Is Possible

Guiding Question: *Infinitely many rules fit the data we have seen. What licenses a machine to prefer one of them?*

Learning as Search

The **hypothesis space** $\mathcal{H}$ is the set of all functions the model family can express — every possible value of the parameters Θ . Learning is the search for the best $h_{\theta} \in \mathcal{H}$.

Exhaustive search

Examine all of $\mathcal{H}$. Feasible only when the space is small (with a few special exceptions).

Heuristic search

Use assumptions to explore only part of the space. Efficient, but the optimum may be missed.

Except for very simple problems you cannot have both: searching **all** hypotheses **efficiently and robustly**.

Inductive Reasoning and Inductive Bias

Inductive reasoning: generalising from seen examples to unseen ones — from observations to a rule that predicts future observations.

Inductive bias: the assumptions by which an algorithm generalises. The choice of hypothesis space is already one; so are assumptions about noise, outliers, the data distribution and the shape of errors.

Without an inductive bias, a system **cannot generalise** at all.

The Bias–Variance Trade-Off

Learning trades **fitting the observed data** against **holding to prior assumptions about unseen data**.

Bias

The tendency to favour particular outcomes regardless of the data — the *stubborn learner*.

High bias → robust to noise, but misses real structure → **underfitting**.

Variance

The tendency to follow irregularities in the data.

High variance → captures patterns, but also noise → **overfitting**.

More parameters generally means more variance. The trade-off is not a defect of particular algorithms — it is a property of inductive inference.

Takeaway

Learning is a **search** through a hypothesis space, steered by an **inductive bias** you chose — knowingly or not. Every model sits somewhere on the bias–variance axis, and no model escapes it.

Fitting h by Repeated Correction

LMS (least mean squares)

Initialise the weights w .

repeat pick a training example $(\mathbf{x}, y)$ compute the prediction $h_w(\mathbf{x})$ compute the error $e = y - h_w(\mathbf{x})$ update every weight: $w_i \leftarrow w_i + \eta e x_i$ **until** convergence on D

What the Update Rule Encodes

Direction

The sign of e decides whether the prediction goes up or down.

Attribution

The factor x_i gives each feature credit proportional to its value.

Step size

The learning rate η trades speed against stability.

Too large an η oscillates or diverges too small an η converges slowly. It is a hyperparameter — chosen, not learned.

Module 3

Linear Regression

Guiding Question: Why squared error — and what exactly does "linear" forbid?

Classification versus Regression

Classification

Y is a finite set of labels.

Will this customer default — yes or no?

Regression

$Y = \mathbb{R}$, the target is a quantity.

What credit line should this customer receive?

The learning machinery is the same only the shape of Y — and therefore the loss — changes.

The Model

Linear regression predicts the target as a weighted sum of the features:

$$h_{\mathbf{w}}(\mathbf{x}) = w_0 + w_1x_1 + \dots + w_px_p = \mathbf{w}^\top \mathbf{x}$$

with $x_0 = 1$ absorbing the intercept.

The hypothesis space is $\mathbb{R}^{p+1}$: one hypothesis per weight vector.

The Objective: Residual Sum of Squares

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w} \in \mathbb{R}^{p+1}} \text{RSS}(\mathbf{w}), \quad \text{RSS}(\mathbf{w}) = \sum_{i=1}^n (y_i - \mathbf{w}^\top \mathbf{x}_i)^2$$

RSS measures **effectiveness** — how good the predictions are. Do not confuse it with **efficiency**, which is about runtime and computational effort.

One Feature: The Closed Form

For a single feature the minimiser can be written down directly:

Slope

$$\hat{w}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

The covariance of x and y over the variance of x .

Intercept

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

The fitted line passes through the point of means.

How Much Variance Is Explained?

$$\text{TSS} = \text{ESS} + \text{RSS}, \quad R^2 = \frac{\text{ESS}}{\text{TSS}} = 1 - \frac{\text{RSS}}{\text{TSS}}$$

where TSS is the total, ESS the explained and RSS the residual sum of squares.

R^2 is the fraction of the target's variation the model accounts for: 0 means no better than predicting the mean, 1 means an exact fit.

Training R^2 never decreases when a feature is added, however useless the feature. A high training R^2 is not evidence that a model generalises — test R^2 and adjusted R^2 can and do fall.

Higher Dimensions

With p features the same objective in matrix form,

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

gives the optimum in one step whenever $\mathbf{X}^\top \mathbf{X}$ is invertible.

Invertibility fails when features are linearly dependent or when $p > n$ — more unknowns than equations. Both are common in practice, and both are why regularisation exists.

Takeaway

Linear regression fits a weighted sum by minimising squared residuals. Its solution is built from **covariance over variance**, and R^2 reports how much variation it explains — not whether it will generalise.

Linear in the Parameters, Not in the Features

A model is **linear** when it is linear in the **weights** $\mathbf{w}$. The features may be arbitrary transformations of the input.

Plain

$$h(\mathbf{x}) = w_0 + w_1x$$

a straight line

Polynomial

$$h(\mathbf{x}) = w_0 + w_1x + w_2x^2$$

a curve — still a *linear model*

Basis functions

$$h(\mathbf{x}) = \sum_j w_j \phi_j(\mathbf{x})$$

any fixed feature map ϕ

Consequences

The good

The closed-form solution and all statistical guarantees carry over unchanged.

The catch

The feature map is a **modelling decision**. Choose too rich a basis and the model overfits too poor and it underfits.

Increasing polynomial degree is a dial on the bias–variance axis you can watch turning.

The Question Nobody Asks

We minimised the *squared* residuals. Why squares — why not absolute deviations, or fourth powers?

Geometric View

The predictions $\mathbf{X}\mathbf{w}$ live in the **column space** of $\mathbf{X}$ — everything the features can express. The observed $\mathbf{y}$ generally does not.

Least squares picks the point of that space **closest to $\mathbf{y}$ in Euclidean distance**: the orthogonal projection. The residual is perpendicular to every feature.

Consequence

The residual carries no remaining linear information about the features.

The Gauss–Markov Theorem

Gauss–Markov (informal). Among all estimators that are

- **linear** in the observed outputs $(y_1, \dots, y_n)$, and
- **unbiased**,

the least-squares estimator has the **lowest variance**. It is the *Best Linear Unbiased Estimator* — BLUE.

Best

lowest variance among the candidates

Linear

a linear function of the observations

Unbiased

right on average over repeated samples

What This Buys, and What It Costs

The payoff

Minimising RSS yields optimal statistical properties under **very mild assumptions** — no normality needed, only uncorrelated errors of constant variance.

The price

Alternatives — absolute error, Huber loss — are **more robust to outliers** but give up the BLUE guarantee.

Squared error is not an arbitrary convenience: it is the loss whose minimiser is the best linear unbiased estimator. Departing from it is legitimate — but it is a trade, and you should be able to name what you traded.

Two Ways to the Same Minimum

Direct (closed form)

$$\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

Exact, one step. Requires forming and inverting a $(p+1) \times (p+1)$ matrix — costly for large p , impossible when it is singular.

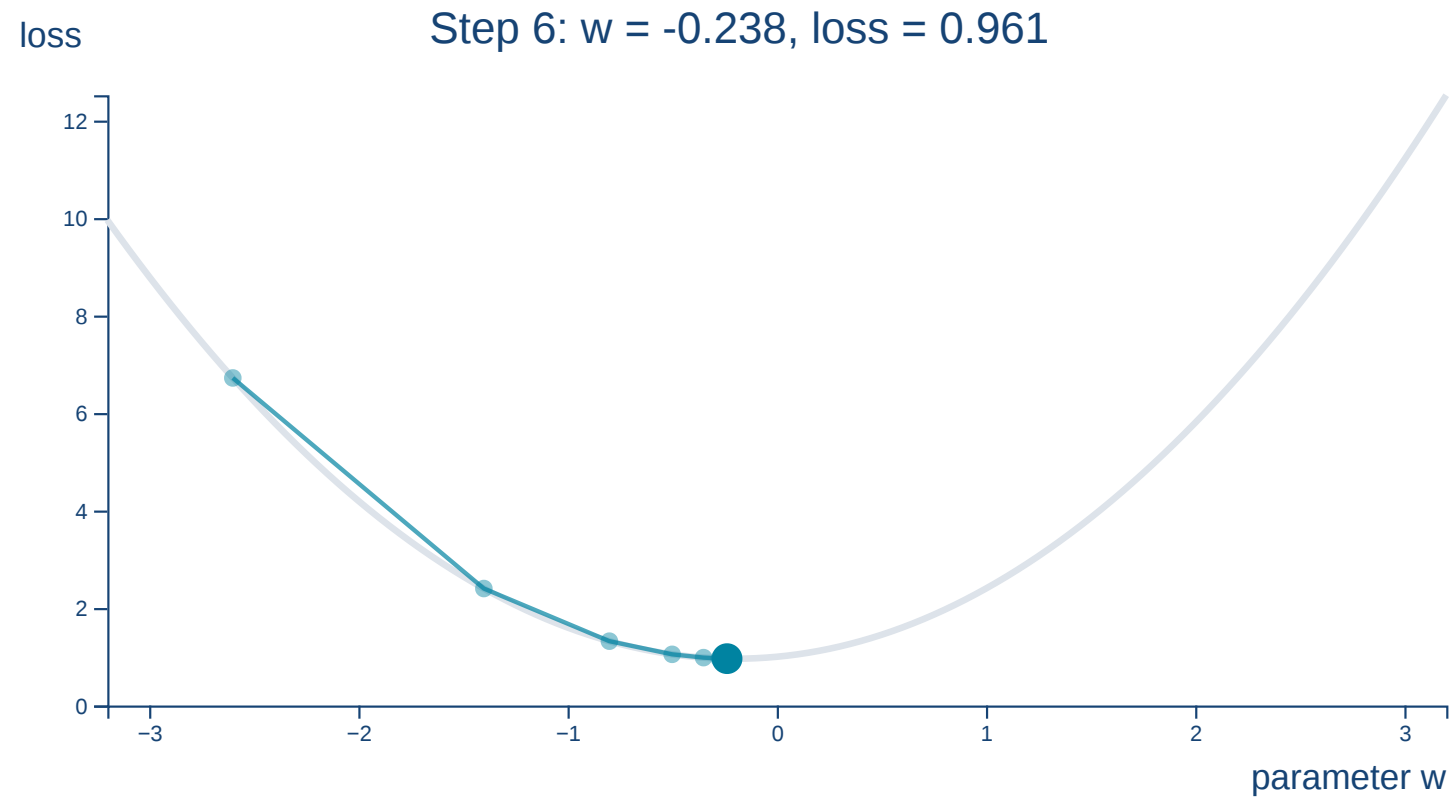
Iterative (gradient descent)

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \text{RSS}(\mathbf{w})$$

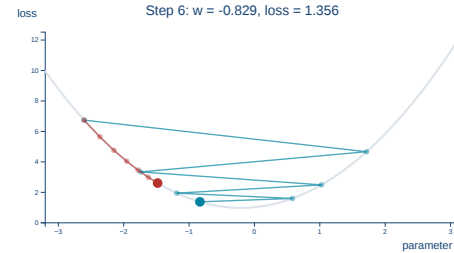
Approximate, many steps. Scales to large p and n , works when the direct solution does not exist, and extends to models with no closed form at all.

The iterative rule is exactly the **LMS algorithm** from Module 2: the update $w_i \leftarrow w_i + \eta e x_i$ is a step along $-\nabla \text{RSS}$, written per example instead of per dataset.

Watching Gradient Descent Work



The Learning Rate Decides Everything



Teal: $\eta = 0.9$ — overshoots and oscillates across the minimum. Red: $\eta = 0.05$ — stable, but barely moves.

The learning rate is a **hyperparameter**: it is chosen, not learned, and both failure modes are silent — one looks unstable, the other looks converged.

Takeaway

Use the **closed form** when the matrix is small and well-conditioned use **gradient descent** when it is not — which, for everything after this unit, is always.

Module 4

From Regression to Classification

***Guiding Question:** A decision is a yes or a no. What goes wrong when we fit a straight line to one?*

Fitting a Line to Labels

Encode the two classes as $y \in \{-1, +1\}$, fit a linear model, and classify by the sign of the prediction:

$$h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

One feature

The **discrimination point** x' satisfies

$$w_0 + w_1 x' = 0$$

Everything to one side is one class.

Two features

The **discrimination line** satisfies

$$w_0 + w_1 x_1 + w_2 x_2 = 0$$

In p dimensions it becomes a hyperplane.

Where It Breaks

Squared loss punishes correctness

A point far on the *correct* side has a large residual, so least squares drags the boundary toward it.

The boundary shifts

Adding well-classified points — or an outlier — moves the discrimination point, even when nothing was misclassified.

The output is also unbounded: $\mathbf{w}^\top \mathbf{x}$ can be 17 or -40 , and neither is interpretable as a degree of belief.

Two Problems, One Fix

$\text{sign}(\mathbf{w}^\top \mathbf{x})$ is not differentiable

No gradient means no gradient descent — the optimisation machinery does not apply.

The boundary is unstable

Squared loss on ± 1 labels lets well-classified points shift the discrimination point.

Replace the hard threshold with the **logistic sigmoid**:

$$h_{\Theta}(\mathbf{x}) = \text{expit}(\mathbf{w}^\top \mathbf{x}) \approx P(y = 1 \mid \mathbf{x})$$

The output becomes a smooth, bounded quantity that can be read as a probability — and can be differentiated.

Odds and Log-Odds

Odds

The ratio of favourable to remaining cases.

Rolling a six: odds 1 : 5.

Logit

The **log-odds** of a probability p :

$$\text{logit}(p) = \ln \frac{p}{1 - p}$$

Range

$p \in (0, 1)$ maps to $(-\infty, +\infty)$
— exactly the range a linear model produces.

The modelling assumption: the logit of the target is a linear combination of the features.

$$\ln \frac{P(y = 1 \mid \mathbf{x})}{1 - P(y = 1 \mid \mathbf{x})} = \mathbf{w}^\top \mathbf{x}$$

The Model

Inverting the logit gives the **logistic sigmoid**:

$$h_{\Theta}(\mathbf{x}) = \text{expit}(\mathbf{w}^{\top} \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^{\top} \mathbf{x}}}$$

Shape

An S-curve: near-linear around 0, saturating towards 0 and 1 at the extremes.

Reading

$h_{\Theta}(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$, and the decision boundary is still $\mathbf{w}^{\top} \mathbf{x} = 0$ — where the sigmoid passes 0.5.

The Loss: Cross-Entropy

Binary cross-entropy (log loss)
$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln h_{\theta}(\mathbf{x}_i) + (1 - y_i) \ln (1 - h_{\theta}(\mathbf{x}_i)) \right]$$

For $|Y| > 2$ classes, the multiclass form sums over the classes with a softmax output.

Where does it come from? From the **likelihood**: $P(y_i = 1 \mid \mathbf{x}_i; \theta) = h_{\theta}(\mathbf{x}_i)$ and $P(y_i = 0 \mid \mathbf{x}_i; \theta) = 1 - h_{\theta}(\mathbf{x}_i)$. Maximising the likelihood of the data is minimising this loss.

Why It Fixes the Problem

Confident and correct

Loss goes to zero — no penalty for being emphatically right.

Confident and wrong

Loss goes to infinity — exactly the errors we want punished.

There is no closed-form solution the parameters are found by gradient descent on this loss.

Takeaway

Logistic regression is a **linear model of the log-odds**, trained by maximising likelihood. It keeps the linear decision boundary but produces a calibrated probability — and a loss that punishes the right kind of error.

Module 5

Probabilistic Classification

Guiding Question: *The data tells us how attributes behave given the class. We need the reverse. What does the conversion cost?*

Thinking With Uncertainty

Bayes' theorem lets us update a **hypothesis** H in the light of **data** D :

$$P(H \mid D) = \frac{P(H) \cdot P(D \mid H)}{P(D)}$$

This is the whole apparatus of evidence-based decision making in one line: a belief, a measurement of how well the belief explains what we saw, and a normalisation.

Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

Likelihood

90% of people with the flu
show headache and coughing

$$P(D \mid H) = 0.9$$

Prior

5% of people currently have
the flu

$$P(H) = 0.05$$

Evidence

headache and coughing occur
in 20% of people

$$P(D) = 0.2$$

How likely is it that you have the flu?

Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

Likelihood

90% of people with the flu
show headache and coughing

$$P(D \mid H) = 0.9$$

Prior

5% of people currently have
the flu

$$P(H) = 0.05$$

Evidence

headache and coughing occur
in 20% of people

$$P(D) = 0.2$$

How likely is it that you have the flu?

$$P(H \mid D) = \frac{0.05 \cdot 0.9}{0.2} = 0.225$$

Why So Low?

Few people have the flu

The prior $P(H) = 0.05$ caps how far the evidence can carry us.

The symptoms are common

$P(D) = 0.2$ means many people have these symptoms *without* the flu.

A symptom is informative to the degree that it is **rare in general** but **common under the hypothesis**. A likelihood alone tells you nothing.

Takeaway

Bayesian decision making weighs **prior × likelihood** against how ordinary the evidence is. Deciding under uncertainty means keeping all three numbers in view, not just the one the study reports.

Two Directions of the Same Connection

Observed connection

$$A_1, \dots, A_p \mid B$$

Given the class, how do the attributes behave? — the standard situation, and the one data can answer directly.

Reversed connection

$$B \mid A_1, \dots, A_p$$

Given the attributes, which class? — the diagnosis situation, and the one we actually need.

$$P(B \mid A_1, \dots, A_p) = \frac{P(B) \cdot P(A_1, \dots, A_p \mid B)}{P(A_1, \dots, A_p)}$$

What the Classifier Needs

Class priors

$$P(\text{class} = y)$$

How common is each class, before seeing any attribute?

Attribute likelihoods

$$P(\text{attribute} = v \mid \text{class} = y)$$

How does each class distribute its attribute values?

Given both, the posterior $P(\text{class} \mid \text{attributes})$ follows from Bayes — and the prediction is the class with the highest posterior.

Estimating $P(A_1, \dots, A_p \mid B)$ requires the **joint** distribution of all attributes within each class: exponentially many parameters, and never enough data.

Generative and Discriminative Classifiers

Generative

Model the **joint** distribution $P(A, B)$ — how the data was produced — then derive $P(B | A)$.

Naive Bayes.

Discriminative

Model $P(B | A)$ **directly** — only the boundary between classes.

Logistic regression.

The Naive Assumption

Assume the attributes are **conditionally independent given the class**:

$$P(A_1, \dots, A_p \mid B) = \prod_{j=1}^p P(A_j \mid B)$$

Exponentially many parameters collapse to p per class — one distribution per attribute.

The assumption is almost always false. Naive Bayes nonetheless classifies well, because ranking the classes correctly is easier than estimating their probabilities correctly.

The Classifier

$$P(B_i \mid A_1, \dots, A_p) = \frac{P(B_i) \cdot \prod_{j=1}^p P(A_j \mid B_i)}{\sum_{i=1}^k P(B_i) \cdot \prod_{j=1}^p P(A_j \mid B_i)}$$

Score each class

prior × product of attribute likelihoods

Normalise

divide by the total over all classes — the theorem of total probability

Predict

the class with the largest posterior

For prediction alone the denominator can be skipped — it is the same for every class. It is needed only when the probability itself is reported.

Example: Enjoy Sport?

Outlook	Temperature	Humidity	Wind	EnjoySport
sunny	hot	high	weak	no
sunny	hot	high	strong	no
overcast	hot	high	weak	yes
rain	mild	high	weak	yes
rain	cool	normal	weak	yes
rain	cool	normal	strong	no

Estimate the priors and the per-attribute likelihoods by **counting**, then score a new day.

The Zero-Frequency Problem

If an attribute value never occurs with a class in the training data, its estimated likelihood is **zero** — and one zero factor drives the whole product to zero, whatever the other attributes say.

Laplace smoothing adds a pseudo-count to every combination, so that unseen does not mean impossible.

One missing count

vetoed every other piece of evidence

Naive Bayes buys tractability with a **conditional independence assumption it knows to be false** — and works anyway, because classification needs the right argmax, not the right probability.

Module 6

Measuring Performance

Guiding Question: *A model reports 99.9% accuracy and has never detected a single case. What should we have measured instead?*

What We Want to Know

The **true misclassification rate** of a hypothesis h is the probability that it errs on a randomly drawn object:

$$\text{err}^*(h) = P(h(\mathbf{x}) \neq y(\mathbf{x}))$$

It is defined over the **whole distribution of objects** — which we cannot observe.

What We Can Measure

We estimate $\text{err}^*(h)$ by the error on a set of examples $D_s \subseteq D$ whose true labels are known:

$$\text{err}(h, D_s) = \frac{1}{|D_s|} \sum_{\mathbf{x} \in D_s} 1[h(\mathbf{x}) \neq y(\mathbf{x})]$$

The estimate is only as good as the sample. Which examples land in D_s — and whether h has seen them before — decides whether the number means anything.

Training Error Is Not an Estimate

When the evaluation set equals the training set, $D_{tr} = D$, the resulting number measures how well the model **memorised**, not how well it will generalise.

Training error is a lower bound on the true error, and an arbitrarily bad one. A model with enough capacity can drive it to zero while learning nothing.

Four Estimation Schemes

Hold-out (2-fold)

Split D into a training and a test part, $D_{tr} \subset D$. Fit on one, evaluate on the other. *Simple, but wastes data and depends on the split.*

k -fold cross-validation

Partition D into k folds; each fold serves once as the test set. Average the k errors. *Uses all data for both roles $k = 5$ or 10 is customary.*

Leave-one-out (n -fold)

$k = n$: every example is a test set of size one. *Nearly unbiased, expensive, and high variance.*

Bootstrapping

Resample D with replacement to build training sets evaluate on the examples left out. *Useful when D is small.*

Misclassification Costs

Not every error costs the same. A missed diagnosis and a false alarm have different consequences, so counting errors equally is itself a modelling assumption.

A **cost matrix** assigns a cost to each (true class, predicted class) pair; the objective becomes expected cost rather than error rate.

Takeaway

An error number is meaningless without the sampling scheme that produced it. **Never evaluate on data the model was fitted on**, and state the cost model when errors are not interchangeable.

A Number Alone Means Nothing

"95% accuracy" is not a result until you know what a trivial method achieves.

Trivial classification

Always predict the **majority class**.

Trivial regression

Always predict a **central tendency** — the mean or median of y .

Random guessing

Draw a label from the marginal distribution $p(y)$.

Example: Why Accuracy Deceives

A screening test for a disease affecting **1 in 1,000** people.

The "model"

Always predict *healthy*.

Its accuracy

99.9% — and it has never detected a single case.

On imbalanced data the majority baseline is nearly unbeatable by accuracy, and completely useless. The metric, not the model, is the problem.

Why the Error Rate Is Not Enough

For multiclass classification $h_{\Theta} : X \rightarrow Y$ with $Y \subseteq \{1, \dots, N\}$, the plain error rate:

Fails on imbalance

The majority baseline scores high while detecting nothing.

Hides the distribution

Which classes the errors fall on stays invisible.

Hides the kind of error

Missing assignments and imprecise assignments count the same.

The Confusion Matrix

Rows are true classes, columns are predictions cell (i, j) counts the examples of class i that h assigned to j . The diagonal is correct, and every off-diagonal cell names a specific confusion.

The matrix is the raw material: every measure below is a ratio of its cells. Read it before reading any summary number.

Four Counts, Per Class

For a class j , every prediction falls into one of four cells:

TP_j — **true positives**

Assigned to j **and** truly in j .

FP_j — **false positives**

Assigned to j but **not** in j .

FN_j — **false negatives**

Not assigned to j but truly in j .

TN_j — **true negatives**

Not assigned to j and not in j .

Read the letters as **decision then correctness**: P/N is what h decided, T/F is whether that decision was right.

Precision and Recall

Recall — completeness

$$\text{rec}_j = \frac{TP_j}{TP_j + FN_j}$$

Of everything that truly belongs to j , what fraction did h find?

Precision — exactness

$$\text{prec}_j = \frac{TP_j}{TP_j + FP_j}$$

When h says j , how often is it right?

Both are **conditional probabilities with different conditions** — recall conditions on the truth, precision on the prediction. Swapping them is the same error as swapping the conditionals in Bayes' theorem.

Mini-Exercise

A spam filter processes 1,000 emails, 100 of which are spam. It flags 80 emails, of which 60 are genuinely spam.

Compute precision and recall. Which one would you rather improve, and why?

Mini-Exercise — Solution

Mini-Exercise — Solution

$$\text{Precision} = 60/80 = 0.75$$

A quarter of flagged mail is legitimate.

$$\text{Recall} = 60/100 = 0.60$$

Forty spam messages still arrive.

For a spam filter, **precision** usually matters more: a lost legitimate email costs far more than a spam message that slips through. The cost matrix decides, not the metric.

Takeaway

The confusion matrix shows **which** errors occur precision and recall split "being wrong" into **being imprecise** and **being incomplete**. Which one you optimise is a decision about costs, not about mathematics.

Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (h(\mathbf{x}_i) - y_i)^2 \quad \text{RMSE} = \sqrt{\text{MSE}}$$

MSE

Average quadratic loss always non-negative, lower is better.

Unit is the **square** of the target's unit — hard to interpret directly.

Large errors are penalised disproportionately.

RMSE

Same ordering as MSE, but back in the **target's own unit** — which is why it is the one usually reported.

Absolute and Relative Error

MAE

$$\frac{1}{n} \sum_i |h(\mathbf{x}_i) - y_i|$$

Easier to interpret than RMSE
treats all errors proportionally.

Mean error

Drop the absolute value to
keep the **direction** — reveals
systematic over- or under-
prediction.

Mean relative error

$$\frac{1}{n} \sum_i \frac{|h(\mathbf{x}_i) - y_i|}{|y_i|}$$

The average error *per unit of target* — comparable across scales.

Normalised RMSE variants aim to make errors comparable between datasets, but are hard to interpret. Prefer a measure whose unit you can name.

Takeaway

Squared measures punish large errors and lose the unit absolute measures stay interpretable signed measures expose bias. Report the one whose **unit and failure mode** match the decision the prediction feeds.